TECNICATURA SUPERIOR EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL

Centro Educativo Paulo Freire — U.E.G.P. N° 165

CLASE 14 — RESOLUCIÓN

K-Means en Python — Notebook comentado

Flujo guiado + resolución de los 3 ejercicios prácticos

Sobre este documento

Simula un notebook de Jupyter: cada celda de código (In) fue ejecutada realmente con scikit-learn, y la salida (Out) que se muestra —números, tablas y gráficos— es el resultado real de esa ejecución, no un valor inventado.

El dataset de clientes usado en el flujo guiado y en el Ejercicio 2 es sintético, generado con semilla fija (random_state=42) para que sea reproducible.

Flujo Guiado: Segmentación de Clientes

Antes de resolver los ejercicios, reproducimos el flujo completo de la guía práctica con un dataset simulado de 200 clientes, para tener kmeans, X_scaled y datos disponibles en el resto del notebook.

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

np.random.seed(42)

# Generamos 4 grupos de clientes simulados (200 en total),
# combinando dos niveles de ingreso y dos niveles de gasto
datos = pd.DataFrame({
    'Ingresos_Anuales': [...],   # ver notebook fuente para el detalle completo
    'Puntaje_Gasto': [...],
})
print(datos.shape)
print(datos.head())
```

Out[1]:

```
(200, 2)
   Ingresos_Anuales  Puntaje_Gasto
0      29690.937231      83.853174
1      21626.274825      33.562400
2      21389.760327      17.803281
3      79376.779049      83.227186
4      69277.570326      20.698021
```

Paso 1 — Estandarización

In [2]:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(datos[['Ingresos_Anuales', 'Puntaje_Gasto']])
```

Paso 2 — Entrenamiento del modelo

In [3]:

```
kmeans = KMeans(n_clusters=4, init='k-means++', n_init=20, random_state=42)
kmeans.fit(X_scaled)
```

Paso 3 — Inspección de atributos

In [4]:

```
print('labels_[:15] =', kmeans.labels_[:15])
print()
print('cluster_centers_ =')
```

```
print(kmeans.cluster_centers_)
print()
print('inertia_ =', kmeans.inertia_)
```

Out[4]:

```
labels_[:15] = [1 3 3 2 0 0 1 2 2 3 1 2 2 1 2]

cluster_centers_ =
[[ 0.98548777 -0.96201147]
 [-0.94666293  0.94135302]
 [ 0.94979559  0.97434609]
 [-0.98862044 -0.95368764]]

inertia_ = 29.13395640132635
```

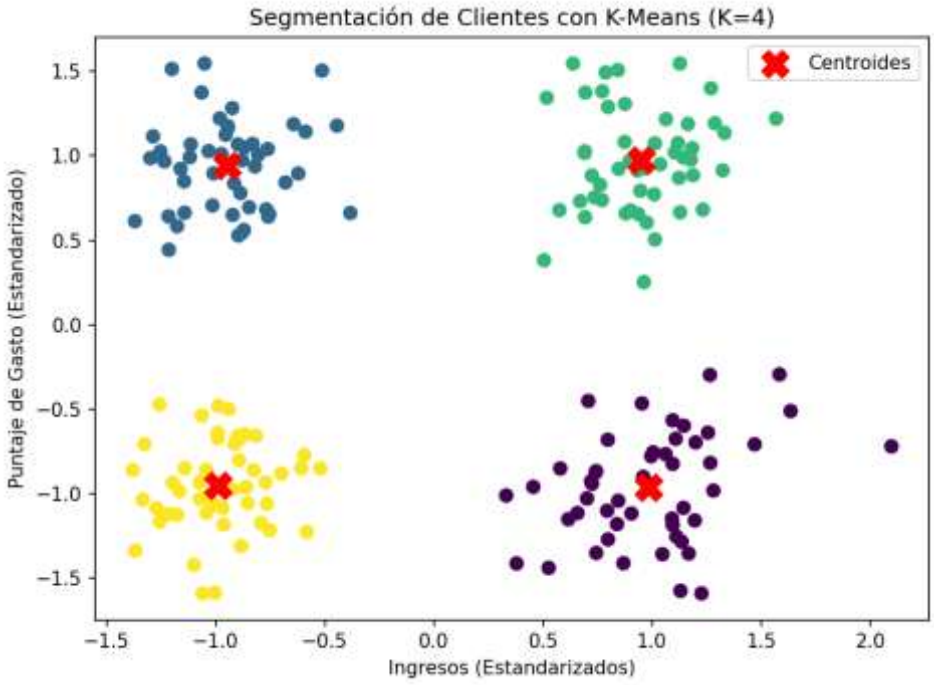
La inercia final es baja (≈29) en relación con la de K=1 (400, ver Ejercicio 2), lo que confirma que 4 clusters compactos describen bien a estos datos — algo esperable, ya que el dataset se construyó combinando 4 grupos bien diferenciados.

Paso 4 — Visualización

In [5]:

```
plt.figure(figsize=(7, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=kmeans.labels_, cmap='viridis', s=45)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            c='red', marker='X', s=200, label='Centroides')
plt.title('Segmentación de Clientes con K-Means (K=4)')
plt.xlabel('Ingresos (Estandarizados)')
plt.ylabel('Puntaje de Gasto (Estandarizado)')
plt.legend()
plt.show()
```

Out[5]:



Resolución de los Ejercicios

EJERCICIO 1 El Impacto de la Inicialización y Óptimos Locales

In [6]:

```
X_blobs, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.5, random_state=3)

# Un único intento de inicialización
modelo_1 = KMeans(n_clusters=3, n_init=1, random_state=3)
modelo_1.fit(X_blobs)
print('Inercia con n_init=1: ', modelo_1.inertia_)

# 20 intentos de inicialización distintos; se conserva el mejor
modelo_2 = KMeans(n_clusters=3, n_init=20, random_state=3)
modelo_2.fit(X_blobs)
print('Inercia con n_init=20:', modelo_2.inertia_)
```

Out[6]:

```
Inercia con n_init=1:  1358.0295293561214
Inercia con n_init=20: 1358.0295293561214
```

Resultado real con estos datos y esta semilla: ambas inercias son idénticas — n_init=1 ya encontró el óptimo global. Esto no contradice la teoría, la explica: KMeans usa por defecto init='k-means++', una inicialización inteligente (no puramente al azar) que ya reduce mucho el riesgo de caer en un óptimo local. Con blobs bien separados como estos, incluso una sola corrida con k-means++ suele bastar.

Respuesta a la pregunta de reflexión

En general, la inercia con n_init=20 es menor o igual que con n_init=1 — nunca puede ser mayor, porque scikit-learn se queda con la mejor de las 20 corridas (o la única, si n_init=1). Que en este caso particular haya dado igual no es un error: significa que la única corrida de n_init=1 ya cayó en el óptimo global, algo más probable cuando los clusters están muy separados y se usa init='k-means++'.

El riesgo real de quedar atrapado en un óptimo local aparece con clusters más solapados o con una inicialización puramente aleatoria (init='random'). Para ilustrarlo de forma más clara, repetimos el experimento con datos más solapados (5 clusters, mayor dispersión) y con init='random' para la corrida única:

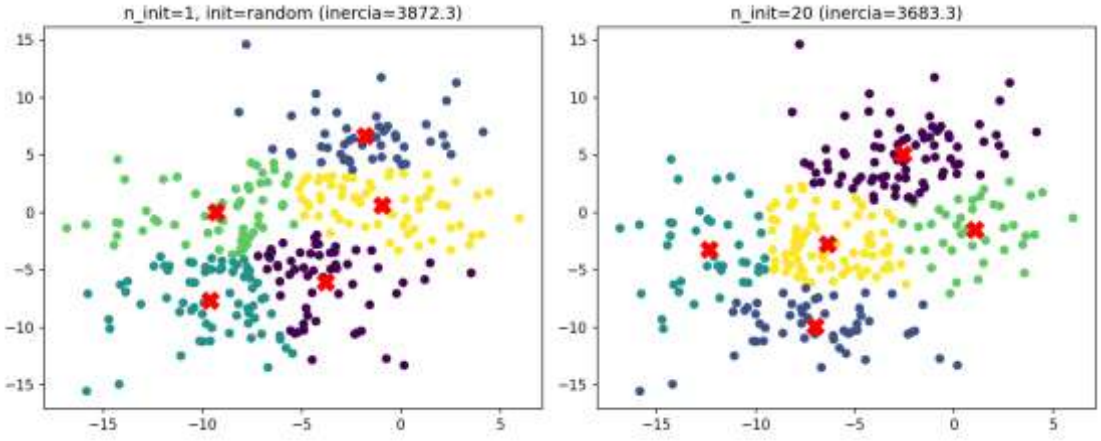
In [7]:

```
# Datos más solapados para forzar un caso donde SÍ se note la diferencia
X2, _ = make_blobs(n_samples=300, centers=5, cluster_std=3.5, random_state=1)

m1 = KMeans(n_clusters=5, n_init=1, random_state=11, init='random').fit(X2)
m2 = KMeans(n_clusters=5, n_init=20, random_state=1).fit(X2)
print('n_init=1, init=random:', m1.inertia_)
print('n_init=20, k-means++ :', m2.inertia_)
```

Out[7]:

```
n_init=1, init=random: 3872.257492689415
n_init=20, k-means++ : 3683.263463572214
```



Acá sí se ve el efecto con claridad: con una sola inicialización aleatoria (izquierda), el algoritmo converge a una partición peor —nótese el cluster amarillo, que mezcla puntos de dos grupos reales— mientras que con 20 intentos (derecha) el algoritmo encuentra una separación más consistente con la estructura real de los datos, y la inercia final es menor.

EJERCICIO 2 El Método del Codo (Elbow Method)

In [8]:

```
inercias = []
rango_k = range(1, 11)

for k in rango_k:
    modelo = KMeans(n_clusters=k, n_init=20, random_state=42)
    modelo.fit(X_scaled)
    inercias.append(modelo.inertia_)

for k, inc in zip(rango_k, inercias):
    print(f'K={k}: inercia={inc:.2f}')
```

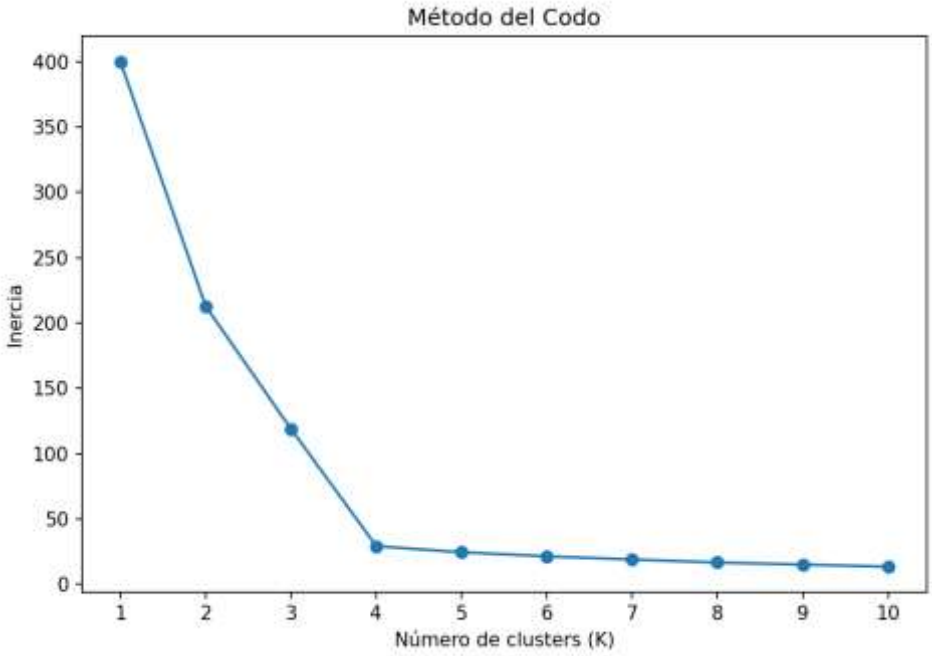
Out[8]:

```
K=1: inercia=400.00
K=2: inercia=212.73
K=3: inercia=118.81
K=4: inercia=29.13
K=5: inercia=24.45
K=6: inercia=21.36
K=7: inercia=18.95
K=8: inercia=16.61
K=9: inercia=15.02
K=10: inercia=13.44
```

In [9]:

```
plt.plot(list(rango_k), inercias, marker='o')
plt.xlabel('Número de clusters (K)')
plt.ylabel('Inercia')
plt.title('Método del Codo')
plt.xticks(list(rango_k))
plt.show()
```

Out[9]:



Análisis

El codo está en K=4: la caída de inercia entre K=1→2→3→4 es muy pronunciada (400 → 213 → 119 → 29), pero de K=4 en adelante la curva se aplana notoriamente (29 → 24 → 21 → 19 → ...) — cada cluster adicional aporta una reducción marginal cada vez menor.

El número óptimo de clusters para este problema de negocio es K=4. Es consistente con cómo se construyó el dataset (4 combinaciones de ingreso alto/bajo y gasto alto/bajo) y con el resultado del flujo guiado, que ya usaba K=4. Elegir un K mayor (por ejemplo 7 u 8) reduciría algo más la inercia, pero a costa de segmentos más difíciles de interpretar en términos de negocio, sin una ganancia proporcional en compacidad.

EJERCICIO 3 ¿Por qué es Vital Escalar las Características?

Respuesta al punto 1 (sin ejecutar código): Salario_Anual varía en un rango de ~$130.000 (entre $20.000 y $150.000), mientras que Hijos varía apenas entre 0 y 5. Como la distancia euclídea eleva al cuadrado la diferencia en cada variable y luego las suma, la diferencia en Salario_Anual entre dos clientes cualesquiera será, en números absolutos, miles de veces mayor que la diferencia en Hijos. En la práctica, Salario_Anual determinará casi el 100% de la distancia, y Hijos quedará prácticamente sin efecto en la asignación de clusters.

In [10]:

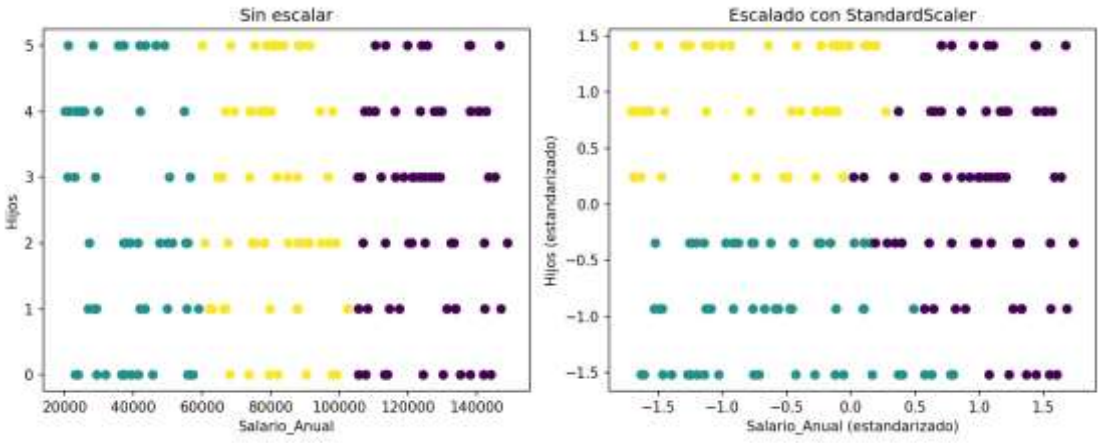
```
np.random.seed(7)
salario = np.random.uniform(20000, 150000, 180)
hijos = np.random.randint(0, 6, 180)
datos3 = pd.DataFrame({'Salario_Anual': salario, 'Hijos': hijos})
X_crudo = datos3[['Salario_Anual', 'Hijos']].values

# --- Sin escalar ---
kmeans_crudo = KMeans(n_clusters=3, n_init=20, random_state=42)
labels_crudo = kmeans_crudo.fit_predict(X_crudo)

# --- Escalado ---
X_std = StandardScaler().fit_transform(X_crudo)
kmeans_std = KMeans(n_clusters=3, n_init=20, random_state=42)
labels_std = kmeans_std.fit_predict(X_std)

fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))
ax[0].scatter(X_crudo[:, 0], X_crudo[:, 1], c=labels_crudo, cmap='viridis')
ax[0].set_title('Sin escalar'); ax[0].set_xlabel('Salario_Anual');
ax[0].set_ylabel('Hijos')
ax[1].scatter(X_std[:, 0], X_std[:, 1], c=labels_std, cmap='viridis')
ax[1].set_title('Escalado con StandardScaler')
plt.show()
```

Out[10]:



Conclusión técnica

El gráfico confirma la predicción del punto 1. Sin escalar (izquierda), los tres clusters son bandas verticales que dependen únicamente de Salario_Anual — Hijos no influye en absoluto en la asignación, a pesar de ser, en principio, una variable igual de relevante para segmentar clientes. Con

StandardScaler (derecha), ambas variables pesan de forma comparable y los clusters resultantes combinan información de Salario_Anual y de Hijos.

Conclusión: en cualquier algoritmo basado en distancia —como K-Means— el preprocesamiento de escala no es opcional. Sin él, las variables con mayor magnitud numérica dominan por completo el resultado, independientemente de su relevancia real para el problema, y las variables de menor escala quedan efectivamente ignoradas.

Cierre

Los tres ejercicios muestran, con código y datos reales, los tres puntos críticos de K-Means: el rol de n_init frente a los óptimos locales, el método del codo para elegir K, y la necesidad de estandarizar variables antes de aplicar el algoritmo.